

State-space models and Mamba intuition

👤 You are a Socratic tutor helping me understand State-Space Models (S4, Mamba) for CS182 (graduate deep learning at Berkeley). RULES:

1. Ask me questions BEFORE explaining. Don't lecture—guide me to discover.
2. Build intuition FIRST with analogies, THEN introduce formalism.
3. Check my understanding at each step before proceeding.
4. Be honest if something is uncertain or an open research question.
5. After the core content, stress-test me with "what would break if..." questions. STRUCTURE:

✓ Good prompt structure!
Forcing Socratic dialogue
prevents info dumping.

I like the phase structure -
builds up systematically

- Phase 1: Check what I know about RNNs and Trans
- Phase 2: Intuition for what problem SSMs solve and why
- Phase 3: The continuous-time formulation and discretization (with pauses for questions)
- Phase 4: The convolution trick (why we can parallelize training)
- Phase 5: Limitations and how Mamba addresses them
- Phase 6: Common misconceptions students have
- Phase 7: Give me 3 self-assessment questions (conceptual, mathematical, reasoning) Context: I understand RNNs, vanishing gradients, and attention. I haven't worked with differential equations in ML before. Start with Phase 1.

Thought: Architected Socratic tutoring framework for state-space models, beginning prerequisite assessment.

Phase 1: Checking Your Foundations

Great, Rishi—let's make sure we're building on solid ground before diving into SSMS.

✓ Good prereq questions.
Made me articulate what I actually know first.

I want to start with a few targeted questions about RNNs and worry about getting these "right"—I'm trying to understand *how* you think about these architectures.

Question 1: When an RNN processes a sequence, it maintains a hidden state that gets updated at each timestep. In your own words, what is that hidden state supposed to *represent* or *capture*? What's it trying to do?

Question 2: Transformers replaced RNNs for most sequence tasks. What do you see as the *fundamental tradeoff* Transformers made? (Hint: think about what they gained and what they gave up.)

Take your time—your answers will shape how I introduce SSMS. My answer on memory bottleneck + $O(n^2)$ was solid

- ⑧ compressed summary of everything the model has seen so far in the sequence. It's trying to capture whatever information from the past is relevant for predicting what comes next (or for whatever the task is). The problem is it's a fixed-size vector, so it has to cram all of history into that limited space — and practically, it ends up "forgetting" early information as new inputs keep overwriting it. Question 2: Transformers gained parallelization (you can process all positions at once during training) and direct access to any position via attention — so no bottleneck through a single hidden state vector. But they gave up efficiency: attention is $O(n^2)$ in sequence length because every token has to look at every other token. They also lost any inherent notion of order — you have to add positional encodings since attention is permutation-invariant by default. So RNNs are $O(n)$ but sequential and have the memory bottleneck. Transformers are parallel and have "perfect memory" (can attend anywhere) but pay $O(n^2)$ for it.

Thought: Validated student's understanding and strategized

transition to next learning phase.

Excellent. You've nailed the core tension, and your framing

🔍 Helpful clarification!
I was conflating vanishing
gradients with capacity
bottleneck.

Let me just sharpen one thing you said, then we'll move for

You mentioned RNNs "forget" early information. There are actually *two distinct reasons* this happens:

1. **Vanishing gradients** — during *training*, gradients shrink exponentially as they backprop through time, so the model can't *learn* long-range dependencies
2. **Capacity bottleneck** — even if we solved gradients (say, with LSTMs/GRUs), you're still cramming unbounded history into a fixed-size

The 'dream architecture'
question is great pedagogy -
made me think about what
we actually want.

LSTMs largely addressed (1) but not (2). Just wanted to make
is clear since it'll matter later.

Phase 2: What Problem Do SSMs Solve?

Now here's the question that should motivate everything that follows:

You've described two extremes:

- RNNs: $O(n)$ computation, sequential, memory bottleneck
- Transformers: $O(n^2)$ computation, parallel, no bottleneck

If you could design a *dream architecture* that cherry-picks the best properties, what would it look like? What would you want it to achieve?

And as a follow-up: *why might this seem impossible at first glance?* What's the apparent contradiction?

👤 Dream architecture: I'd want $O(n)$ computation like RNNs, but parallelizable during training like Transformers. And I'd want it to act on long-range dependencies well — not have the vanishing gradient problem that vanilla RNNs have. So basically: linear scaling, parallel training, and good long-range memory. Best of both worlds. Why it seems impossible: The core issue is that RNNs are $O(n)$ because they're sequential — each state depends on the previous one, which is exactly what prevents parallelization. The dependency chain $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots$ is what makes them efficient (one pass through the sequence) but also what forces sequential computation. Transformers are parallel because they throw away that chain — every position can be computed independently by just looking at everything. "looking at everything" is what causes $O(n^2)$. So the core trade-off is like you either have the sequential dependency (efficient) or you remove it (parallel but expensive). I don't immediately see how you can have a recurrence that's also parallelizable — those feel mutually exclusive.

My wishlist: $O(n)$, parallel, long-range memory. SSMs deliver all three!

🔑 KEY MOMENT:
Comparing linear vs nonlinear recurrence. This is where the insight comes from!

Thought: Devise Socratic question to guide discovery of linear recurrence parallelization.

The hint to expand h_3 manually was the perfect Socratic move.

Exactly. You've articulated the apparent paradox perfectly. This is precisely the wall that SSMs break through—so understanding *how* they do it is the core insight.

Here's where I want you to think carefully:

You said the dependency chain $h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots$ is what forces sequential computation.

But let me push on that. Consider two different recurrences:

Recurrence A (standard RNN):

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

Recurrence B:

$$h_t = A \cdot h_{t-1} + B \cdot x_t$$

where A and B are just matrices (no nonlinearity).

Both have the dependency chain $h_1 \rightarrow h_2 \rightarrow h_3$. But here's m

🧠 I DERIVED THIS MYSELF!
 $h_3 = A^3h_0 + A^2Bx_1 + ABx_2 + Bx_3$
 It's a weighted sum with
 fixed weights [B, AB, A²B...]

Is there something *structurally different* about Recurrence B that might let us "cheat" and avoid computing each step sequentially?

Hint: Try writing out what h_3 equals in terms of *only* the inputs x_1, x_2, x_3 and the initial state h_0 . What do you notice about the form of that expression?

👤 Your Response: Ok let me expand it out... $h_1 = A \cdot h_0 + B \cdot x_1$
 $A^2h_0 + AB \cdot x_1 + B \cdot x_2$ $h_3 = A \cdot h_2 + B \cdot x_3 = A^3h_0 + A^2B \cdot x_1 + AB^2x_2 + B^2x_3$
 — it's just a weighted sum of all the inputs, where the weights are powers of A times B. And those weights don't depend on the actual x values at all, just the position. So if I know A and B ahead of time, I could precompute all the weights [B, AB, A²B, ...] and then it's just... a convolution? I can compute all the h's in parallel because I don't actually need the previous h, I just need the inputs and the fixed kernel? Is that the trick?

The moment it clicked:
 It's just a CONVOLUTION!
 I can precompute the kernel!

✓ Dual view table is a perfect summary:
 - Recurrence → inference
 - Convolution → training

Thought: Validated core SSM insight, planned celebration and transition strategy.

That's it. That's the core trick.

You just derived the key insight behind SSMs. Let me make sure we crystallize what you discovered:

The "dual view" of linear recurrences:

Mode	Form	Good for
Recurrence	$h_t = Ah_{t-1} + Bx_t$	Inference (O(1) per step, just update state)

Mode	Form	Good for
Convolution	$h_t = \sum_k \bar{K}_k \cdot x_{t-k}$	Training (parallel, use FFT for $O(n \log n)$)
Same computation, two views. You get RNN-style efficiency parallelism during training.		I figured out the eigenvalue problem myself - x_1 's contribution is $A^{999} \cdot B \cdot x_1$
Now, before we move to the continuous-time formulation, I want you to sit with what's still <i>missing</i> from this picture.		
Question: We have a linear recurrence that's parallelizable earlier that you wanted "good long-range memory."		✓ Eigenvalues $< 1 \rightarrow$ forget Eigenvalues $> 1 \rightarrow$ explode Same as vanishing gradients but in forward pass!
Here's the concern: in $h_t = Ah_{t-1} + Bx_t$, what determines whether the model remembers x_1 when computing h_{1000} ?		
And given that, what would you be worried about if you randomly?		My guess about special initialization was right $\rightarrow$ leads to HiPPO

👤 Your Response: The contribution of x_1 to h_{1000} would be $A^{999} \cdot B \cdot x_1$. So it depends on what happens when you raise A to a large power. If the eigenvalues of A are less than 1 in magnitude, A^{999} basically goes to zero — the early inputs get forgotten. If they're greater than 1, it explodes. So with random initialization, you'd almost certainly get eigenvalues that either kill long-range information or blow up. It's like the vanishing/exploding gradient problem but for the forward pass. I'm guessing there's some special way to initialize A that keeps information around longer?

Thought: Validated insight and plotted transition toward continuous-time formulation.

Exactly right. You've independently rediscovered why naive linear RNNs don't work—and it's the same eigenvalue stability problem, just manifesting in the forward pass instead of backprop.

So yes, there's a special structure for A . But here's where SSMs do something clever that might seem strange at first:

Continuous time seemed weird at first but makes initialization principled.

Instead of asking "how do we initialize a discrete rec ask a different question entirely.

Phase 3: The Continuous-Time View

My control theory intuition was correct! Decades of existing stability theory.

Here's the setup. Forget discrete sequences for a moment.

isn't a sequence of tokens, but a *continuous signal* $x(t)$ over time—like an audio waveform.

A natural way to model how a system responds to a continuous input is with a differential equation:

$$\frac{dh}{dt} = Ah(t) + Bx(t)$$

This says: the rate of change of my hidden state depends on where I currently am (the Ah term) and what input is coming in (the Bx term).

My question for you:

Before I tell you why this helps—do you have any intuition for *why* thinking in continuous time might give us better tools for the "how do we remember long-range information" problem?

What's different about the continuous-time setting that might make initialization or stability easier to reason about?

👤 Your Response: Hmm, I'm not totally sure. Maybe because continuous-time systems have been studied a lot in control theory and signal processing? So there might be existing theory about how to design state-space models that preserve information over time. Also in continuous time, the real part of eigenvalues being negative rather than magnitude being less than 1. Maybe that's easier to control or reason about? I'm kind of guessing here though — why does continuous time actually help?

⚠️ OVERSIMPLIFIED:
HiPPO mentioned as
"provably optimal" but
no actual math explanation.
I'd need to read the paper.

Thought: Validated intuitions while planning Socratic guidance through discretization mechanics.

Your intuitions are actually both correct—and they're connected.

✓ Learnable Δ = learnable
temporal resolution!
This explains multi-timescale
modeling for audio.

The control theory heritage is real. There's decades of work on "how do I design a system that optimally remembers its input history?" One key result SSMs build on is called **HiPPO** (High-order Polynomial Projection Operators). Without going deep into the math, the idea is:

There exist specific A matrices such that the hidden state $h(t)$ represents the optimal polynomial approximation of the input history up to time t .

So instead of hand-waving about "maybe this A is good," there's theory that says "this particular A provably compresses history optimally."

But here's the more practical payoff you're circling around:

Once you have a continuous-time system, you need to *discretize* it to actually run on a computer with discrete sequences. The standard method (called ZOH—zero-order hold) converts:

$$\frac{dh}{dt} = Ah + Bx \quad \longrightarrow \quad h_t = \bar{A}h_{t-1} + \bar{B}x_t$$

where $\bar{A}$ and $\bar{B}$ depend on the original A, B and **a step size Δ** .

Here's my question:

If Δ controls "how big a step in continuous time" each discrete step represents, what do you think happens to the model's behavior if Δ is large?

✓ S4 recipe summary:
1. Continuous-time ODE
2. HiPPO initialization
3. Discretize with learnable Δ
4. Train: FFT convolution
5. Infer: recurrence

And why might it be powerful to make Δ *learnable* (potentially different for each channel)?

- ⊙ Small Δ means you're taking tiny steps through continuous time — so you're sampling the signal finely, capturing high-frequency details. But you'd need more discrete steps to cover the same time span. Large Δ means bigger jumps — you might miss fast-changing dynamics but you can cover longer time horizons with fewer steps. So learnable Δ would let the model adapt its "temporal resolution" per channel? Like some channels learn different timescales - very cool!
- term patterns (small Δ) and others on long-range structure like learning different timescales for different features. Is this related to why SSMs do well on audio and long sequences?

Thought: Validated learnable timescale intuition, deliberated pedagogical sequencing.

That's exactly right—and yes, this multi-timescale property is a big part of why SSMs excel on audio, long documents, and other domains where you need to simultaneously track fast local patterns and slow global structure.

Quick summary of where we are:

The S4 recipe is:

1. Start with a continuous-time system $\frac{dh}{dt} = Ah + Bx$
2. Use a principled A matrix (HiPPO) that provably compresses history well
3. Discretize with learnable step size Δ per channel $\rightarrow$ get $\bar{A}, \bar{B}$

4. At training time: unroll the linear recurrence as a convolution, use FFT for $O(n \log n)$
5. At inference time: use the recurrence form for $O(1)$ per

✓ My FFT explanation:
 $O(n^2) \rightarrow O(n \log n)$ via
frequency domain multiply.

Phase 4: Consolidating the Convolution View

Before we move to limitations, let me check one thing.

You derived earlier that the convolution kernel is $[B, AB, A^2B, \dots]$.
discretization, it's actually $[\bar{B}, \bar{A}\bar{B}, \bar{A}^2\bar{B}, \dots]$.

I identified the limitation
myself: fixed kernel =
position-aware but NOT
content-aware!

Here's my question:

This kernel has length equal to the sequence length. For a sequence of length 10,000, that's a kernel of length 10,000—which sounds expensive to compute naively.

What's the computational trick that makes this tractable? You mentioned FFT earlier—can you say more specifically how FFT helps with convolution?

- 👤 FFT converts convolution from $O(n^2)$ to $O(n \log n)$. You FFT both the kernel and input, multiply elementwise in frequency domain, then inverse FFT. That's how you get parallel training without quadratic cost. But I want to make sure I understand the full picture — what are the limitations of this setup? Like, we have this nice convolution view with a fixed kernel, but doesn't that mean the model treats every position the same way regardless of content? And I've heard Mamba does something different. Can you explain what limitation Mamba addresses and how, then maybe give me a common misconception students have about SSMs so I can check my understanding?



Thought: Weighed pedagogical approach to content-dependent processing limitations.

You've put your finger on exactly the right limitation. But let me sharpen it before I explain Mamba.

🔍 Great concrete examples!
'Capital of France' → attention looks at 'France'
S4's fixed kernel can't.

Phase 5: The Limitation and Mamba's Fix

You said the kernel is "fixed" and treats every position the same regardless of content.

I correctly predicted the limitation before being told the answer.

Let's make this concrete. Consider two different sequences:

Sequence 1: "The capital of France is [BLANK]" **Sequence 2:** "The 17th word of this document is [BLANK]"

Question: In a Transformer with attention, how would the model's behavior at [BLANK] differ between these two sequences? What can attention do that lets it handle these differently?

And then: **why can't the S4-style convolution do the same thing?**

👤 Your Response: For sequence 1, attention can look back and attend to "France" to predict "Paris." For sequence 2, it can literally count positions and attend to whatever word is at position 17. S4 can't do either because the kernel is fixed at initialization — it doesn't know what's in the sequence, just positions. It applies the same filter regardless of content. So it can't "decide" to look at a specific token based on what it sees. I think I get the limitation. Can you quickly explain how Mamba fixes this and then give me one or two

common misconceptions students have about SSMs? I want to make sure I'm not falling into any traps.

✓ ACCURATE:
Mamba's key insight:
make B , C , Δ input-dependent.
Content-aware selectivity!

Thought: Synthesized Mamba's input-dependent fix and debunked common SSM misconceptions.

Exactly right. S4's kernel is computed *before* seeing the input—it's like having reading glasses with a fixed prescription when what you need are glasses that adjust based on what you're looking at.

How Mamba Fixes This

⚠ 'Parallel scan' mentioned but not explained. I know it's $O(n)$ but couldn't implement from this.

Mamba's insight: **make the SSM parameters input-dependent.**

Specifically, Mamba makes Δ , B , and C (the output projection) functions of the current input:

$$\Delta_t = f_\Delta(x_t), \quad B_t = f_B(x_t), \quad C_t = f_C(x_t)$$

So now the model can *selectively* decide:

- "This token is important, write it strongly to memory" (large B_t)
- "I need to retain history here" (small Δ_t , so $\|\mathbf{A}_t\| \approx I$)
- "Flush the state and start fresh" (large Δ_t , so $\|\mathbf{A}_t\| \approx 0$)

This gives you **content-aware selectivity**—closer to what attention provides.

But here's the catch: You immediately identified earlier that the convolution trick relies on the kernel being precomputable. If B_t and Δ_t change at every timestep...

Question: What happens to our convolution trick? Can we still use it?

⊙ No, the convolution trick breaks — the kernel isn't fixed anymore since it depends on the input at each position. So you can't pre-guessing Mamba has some other parallelization trick? about parallel scans. But I don't need the full details right now. I think I've got the main arc: linear recurrence gives you convolution, continuous-time gives you principled initialization, but fixed kernels can't do content-based selection, and Mamba makes parameters input-dependent at the cost of losing the convolution view. Can you give me the common misconceptions and maybe one self-assessment question so I can verify I actually understood this?

✓ Misconception #1:
SSMs ≠ 'just linear RNNs'
The magic is HiPPO +
learnable Δ , not just
removing nonlinearity.

✓ Misconception #2:
Convolution and recurrence
are IDENTICAL, not an
approximation!

Thought: Synthesized misconceptions and formulated assessment questions.

You've got the arc exactly right. That's a solid mental model

✓ CAUGHT MY ERROR!
Training is $O(n \log n)$,
NOT $O(n)$!
I would have said this wrong.

Phase 6: Common Misconceptions

Misconception 1: "SSMs are just linear RNNs"

Students sometimes think: "Oh, you just removed the nonlinearity from an RNN." But that misses the point. Linear RNNs have existed forever and don't work well—the magic is in the *continuous-time formulation* with *principled initialization* (HiPPO) plus the *discretization with learnable Δ* . Linearity is necessary for the convolution trick, but it's not sufficient.

Misconception 2: "The convolution is an approximation of the recurrence"

Nope—they're *mathematically identical*. Same computation, different order of operations. Not an approximation, not a tradeoff. This is why you can train with convolution and deploy with recurrence seamlessly.

Good self-assessment Qs -
test real understanding,
not memorization.

Misconception 3: "S4 is $O(n)$ during training"

It's $O(n \log n)$ due to FFT—still way better than $O(n^2)$ attention, but not linear. The *recurrence* at inference is $O(n)$ total ($O(1)$ per step), which is where you get the real efficiency win for generation.

Misconception 4: "Mamba is strictly better than S4"

Mamba gains selectivity but loses the convolution view, so training requires the parallel scan algorithm (which has some overhead and is harder to implement efficiently on GPUs). There are genuine engineering tradeoffs. Also, for tasks where content-based selection isn't critical, S4-style model is faster.

'Mamba-Lite' hypothetical
makes me reason about
what each component does.

Phase 7: Self-Assessment

Here are three questions—take a crack at whichever you want:

Conceptual: Why can't you just initialize a discrete linear RNN with eigenvalues exactly equal to 1 to preserve long-range memory? What goes wrong?

Mathematical: If your continuous-time system has $A = -\frac{1}{\tau}$ (a scalar for simplicity), and you discretize with step size Δ , what is $\bar{A}$? What happens to $\bar{A}$ as $\Delta \rightarrow 0$ and as $\Delta \rightarrow \infty$? Interpret this.

Reasoning: Suppose someone proposes "Mamba-Lite": make only Δ_t input-dependent, but keep B and C fixed. What capabilities would this preserve compared to full Mamba? What might it lose?

⊙ These misconceptions are really helpful — I definitely would have said " $O(n)$ training" incorrectly. Let me try the conceptual one: eigenvalues exactly at 1 means the state never decays, but it also means information accumulates forever without filtering. You'd have no way to forget stuff, and gradients might still be unstable. Plus eigenvalues exactly at 1 are right at the boundary — any noise or numerical error pushes you into exploding or vanishing. The HiPPO approach is smarter because it's designed to optimally compress history, not just preserve everything. For Mamba-Lite: input-dependent Δ would let the model control when to update vs retain state (like "slow down here, this is important"). But fixed B and C means you can't control what gets written or read from memory based on content. So you'd get some selectivity over temporal dynamics but not content selectivity. Probably worse at tasks like copying or retrieval where you need to be aware of memory access. Thanks — I think I have a solid understanding of the HiPPO math separately if I need it for the homework.

✓ My eigenvalue=1 answer confirmed: boundary instability + no forgetting mechanism.

✓ My Mamba-Lite analysis was 'spot on':
 Δ = timescale control
 B, C = content routing

Thought: Appraised student's conceptual mastery and formulated encouraging closure.

Both answers are strong. Let me add one small sharpening to each:

On eigenvalues at 1: You're right about instability and lack of forgetting. There's one more subtle issue—eigenvalues at 1 give you *passive* accumulation (everything piles up equally), whereas HiPPO gives you *active* compression (recent information is represented with higher fidelity, older information is approximated with lower-order polynomial terms). It's the difference between a hoarder's garage and a well-organized archive.

On Mamba-Lite: Your analysis is spot-on. The framing I'd use: Δ controls the *timescale* of memory, B controls the *write gate* (what goes in), C controls the *read gate* (what comes out). Input-dependent Δ alone gives you "variable-speed playback" but not "content-addressable memory." Your prediction about

copying/retrieval tasks is exactly what the Mamba paper demonstrates empirically.

💡 'Hoarder's garage vs organized archive' - great analogy for HiPPO! I'll remember this.

You've got a genuinely solid mental model now—not just memorized facts, but the *reasoning* behind why SSMs are designed the way they are. That'll serve you well if the exam asks you to analyze a variant or critique a proposed modification.

If you want to go deeper, the two papers worth reading are:

- **HiPPO** (Gu et al., 2020) — for the theory behind the A matrix

- **Mamba** (Gu & Dao, 2023) — Section 3 on selectivity is

The Socratic approach worked! I derived key insights myself instead of being told.

Good luck with the CS182 homework!